ADR 0002 — Authentication: password hashing, sessions, and role assignment

- **Status:** Proposed — awaiting team agreement
- **Date:** 2026-09-22
- **Author:** Nicholas Caesar (`nadcaesar`)
- **Drives:** #16 — F1.5, Move authentication server-side with hashed passwords
- **Depends on:** ADR 0001 (backend stack, #33) · #13 F1.2 schema · #14 F1.3 API service
- **Consumed by:** #15 F1.4 repository seam · F19 role-based access control
- **Supersedes:** the client-side authentication generated into the prototype by Lovable

---

Context

The prototype authenticates entirely in the browser. Three defects follow from that, all verified against the working tree:

1. **Passwords are stored in plaintext.** `AppUser.password` (`src/data/appDatabase.ts:9`) is serialized into `localStorage['flyer_db_v1']`, so every account's password is readable from the DevTools Application panel. Two seed accounts ship with the literal password `password123` (`:62`, `:72`).
2. **Passwords are compared with `===`.** `authenticateUser` (`src/data/appDatabase.ts:187`) does a plaintext string comparison. There is no hashing anywhere in the repository.
3. **Authorization is a client-writable string.** `ProtectedRoutes` (`src/App.tsx:29-30`) renders on `auth.isLoggedIn`, read from `localStorage['flyerrr_auth']` (`src/contexts/AuthContext.tsx:25`). One console paste grants officer privileges:

   ```
   localStorage.setItem('flyerrr_auth', '{"isLoggedIn":true,"role":"org_officer"}')
   ```

4. **Users choose their own role at sign-up.** The login screen offers a Student / Organization Officer choice (`src/pages/Login.tsx:66`, `:80`) and passes the selected value unvalidated into `createUser` (`:29`). Self-registration as `org_officer` requires no DevTools at all. This defect was not in the original issue body; it was found while planning F1.5 and has been added to #16.

This is a privacy and ethics exposure rather than a code-quality problem. Students reuse university passwords, and the organization verification queue — an administrative trust decision under US-08 — is currently protected by an editable string.

Decision drivers

- The acceptance criteria on #16 demand hashing, no client-side credentials, a working **revocation** path, server-side role enforcement, and a test proving that tampering with client state grants no access.

- CI pins Node 20 (`.github/workflows/ci.yml`). Anything requiring a native build toolchain is a CI risk on a team that has never had green CI.
- Four students, a ten-week semester, and no prior backend experience on the team. ADR 0001 commits to "boring and well-documented over novel." This ADR inherits that.
- F19 (role-based access control) is owned by another team member and must consume whatever this ADR produces. The session claim shape is therefore a public contract, not an implementation detail.

Decision 1 — Hash passwords with Argon2id

Chosen: Argon2id via a library shipping prebuilt binaries, with `bcrypt` at cost factor 12 as the fallback if prebuilt binaries are unavailable on the chosen host. Verify the exact package and version at install time; do not copy a version number out of this document.

Option	Assessment
Argon2id, prebuilt binaries	**Chosen.** Current OWASP recommendation. Memory-hard, so it resists GPU attack better than bcrypt. Prebuilt binaries avoid needing a compiler in CI.
Argon2id, compiled from source	Rejected. Same algorithm, but a `node-gyp` build step is a real CI failure mode for a team that needs CI to be boring.
bcrypt, cost 12	**Accepted fallback.** Explicitly permitted by the acceptance criteria and entirely defensible. Weaker against GPU attack than Argon2id, and silently truncates input past 72 bytes — cap password length in validation if this path is taken.
PBKDF2 via Web Crypto	Rejected. Zero dependencies and available everywhere, which is attractive, but it is the weakest of the three against hardware attack and would need a written justification we cannot give.
Plaintext or a bare hash such as SHA-256	Rejected. A single unsalted hash is not password storage, and this is the defect we are fixing.

Consequences: hashing and verification live in exactly one module, `server/src/lib/password.ts` , exporting `hash(plain)` and `verify(plain, hash)` . That file is the only place in the entire codebase permitted to see a plaintext password. Login performs a verification against a dummy hash even when the email is unknown, so response timing does not reveal whether an account exists.

Decision 2 — Opaque session identifiers in a database table, not JWTs

Chosen: On login, generate a cryptographically random session identifier, store a row in a `sessions` table (`id` , `user_id` , `created_at` , `expires_at` , `revoked_at`), and return the identifier in a cookie. Every authenticated request looks the session up.

Option	Assessment
Opaque session ID + `sessions` table	**Chosen.** Revocation is `DELETE FROM sessions WHERE id = $1`. Logout genuinely invalidates. A stolen identifier is meaningless on its own.
Short-lived JWT	Rejected despite being permitted by the acceptance criteria. A JWT cannot be revoked before expiry without a server-side denylist — which is a sessions table with extra cryptography on top. It also tempts us to put `role` in the token, which means a demoted officer keeps their privileges until expiry.
JWT access token + refresh token rotation	Rejected as over-engineered for a campus app with no scale requirement. It is the right answer for a distributed system and the wrong answer for a ten-week capstone.
Signed cookie holding the user object	Rejected. Same revocation problem as a JWT, with a larger cookie.

Consequences: one database round trip per authenticated request, which is irrelevant at this scale. Expired sessions must be swept — do it lazily on lookup plus a periodic cleanup, and document the sweep so it is not mistaken for dead code.

Decision 3 — Cookie policy

Chosen: `httpOnly; Secure; SameSite=Lax; Path=/`, with a 7-day absolute expiry and no sliding renewal.

- `httpOnly` is non-negotiable: it is what makes the "no credential in client storage" criterion true, because JavaScript cannot read the cookie.
- `Secure` requires HTTPS, which staging must therefore provide.
- `SameSite=Lax` is correct **only if the client and API share a site**. If F1.3 deploys the API to a different origin than the SPA, this must become `SameSite=None; Secure` *and* the API must send `Access-Control-Allow-Credentials: true` with an explicit origin allowlist, while the client sends `credentials: "include"`. **This is an open question for F1.3 and is tracked as coordination item 2 on #16.**
- Absolute rather than sliding expiry, because a sliding window means a stolen session can be kept alive indefinitely.

Decision 4 — Roles are assigned by the server, never requested by the client

Chosen: `POST /api/auth/signup` ignores any `role` in the request body and always creates a `student`. The `org_officer` role is granted only by an existing officer or admin, through the organization-verification flow (F17 / F18).

Option	Assessment
Server always creates `student`; officer granted by approval	**Chosen.** Matches US-08, which states that an administrator reviews and approves organization leaders *before* they can post.
Trust the client's `role`, as today	Rejected. This is defect 4 above.
Infer the role from an email domain pattern	Rejected. Howard issues the same domain to students and officers, so this would not work, and domain checks are trivially spoofed at sign-up if email is never verified.
Officer sign-up creates a pending request	Deferred, not rejected. This is the better long-term design and belongs to F17/F18. F1.5 only needs to stop the escalation.

Consequences: the role selector on the login screen becomes sign-in-only presentation, or is removed. There must be a documented path for creating the first officer — a seeding script, not a UI.

Decision 5 — Delete the existing plaintext accounts rather than migrate them

Chosen: Drop the two seed users. Provide `server/scripts/seed-dev-users.ts` that reads passwords from environment variables and refuses to run when `NODE_ENV=production`.

Migrating is the textbook answer, so the reasoning for not doing it is recorded deliberately: no real person has an account, the only two records are demo fixtures with a published password, and a rehash-on-next-login path would be roughly as much code as the login endpoint itself while protecting nothing. Writing a migration here would be ceremony. This satisfies the "existing seed accounts migrated **or reset**" criterion on the reset branch, and removes `password123` from the repository.

Out of scope for this ADR

Named explicitly so reviewers do not ask for them:

- **Second authentication factor.** The working document's F7 is "Dual Authentication Sign-In," and `docs/backlog.csv` splits the second factor into `F07-03`. This ADR covers the primary factor only. A follow-up ADR should cover TOTP enrollment and recovery.
- **Password reset and email verification.** Both need an outbound mail path, which is F22 and currently unassigned.
- **The full permission matrix.** F19's concern. This ADR supplies the identity; F19 decides what it may do.
- **OAuth or Howard single sign-on.** Not in the backlog. Worth naming as future work because it would eliminate password storage entirely, which is the strongest possible version of this fix.

Consequences

Positive. Plaintext passwords leave the codebase. Logout becomes real. Role claims become trustworthy, which is what makes US-07, US-08, and US-09 demonstrable rather than merely visible. F19 gets a defined contract to build on instead of inventing one.

Negative. The client can no longer determine its own logged-in state synchronously during render, so `ProtectedRoutes` needs a loading state while `GET /api/auth/me` resolves — a visible behavior change in every protected route. The offline localStorage adapter can no longer authenticate at all; it stores the safe user DTO only. And F1.5 now formally depends on F1.3 having a running process, which is the schedule risk documented in `docs/F1.5_SERVER_AUTH_PLAN.md` §8.

Neutral. One database round trip per authenticated request.

How this will be verified

Each item maps to a test named in `docs/F1.5_SERVER_AUTH_PLAN.md` §Phase 4.

- The same password hashes to two different values; `verify` accepts the right password and rejects the wrong one.
- No response body and no client storage entry ever contains `password` or `password_hash`; `grep -rn "password" src/` shows no plaintext storage or comparison.
- A sign-up request carrying `role: "org_officer"` produces a `student` account.
- Unknown-email and wrong-password failures return a byte-identical generic message, and the 6th attempt inside the rate-limit window is rejected.
- Logout removes the session row, and the cookie no longer authenticates.
- **The bypass test:** with `flyerrr_auth` set to `{"isLoggedIn":true,"role":"org_officer"}` and no valid cookie, an officer-only endpoint returns 401 or 403 and the database is unchanged.